4. Image Convolution

Table of Contents

1. Libraries
2. Simple Example
3. PyTorch Convolution

Importing Libraries

In [ ]:
```
import matplotlib.pyplot as plt
import numpy as np
from PIL import Image
```

Simple Convolution

Definition

- **I**: Image to convolve.
- **H**: filter matrix to convolve the image with.
- **J**: Result of the convolution.

The following graphics shows exemplary the mathematical operations of the convolution. The filter matrix **H** is shifted over the input image **I**. The values 'under' the filter matrix are multiplicated with the corresponding values in **H**, summed up and writen to the result **J**. The target position is usually the position under the center of **H**.

In order to implement the convolution with a block filter, we need two methods. The first one will create the block filter matrix **H** depending on the filter width/height **n**.

A block filter holds the value $\frac{1}{n \cdot n}$ at each position:

In [ ]:
```
def block_filter(n):
    H = np.ones((n, n)) / (n * n) # each element in H has the value 1/(n*n)
    return H
```

We will test the method by creating a filter with `n = 5` :

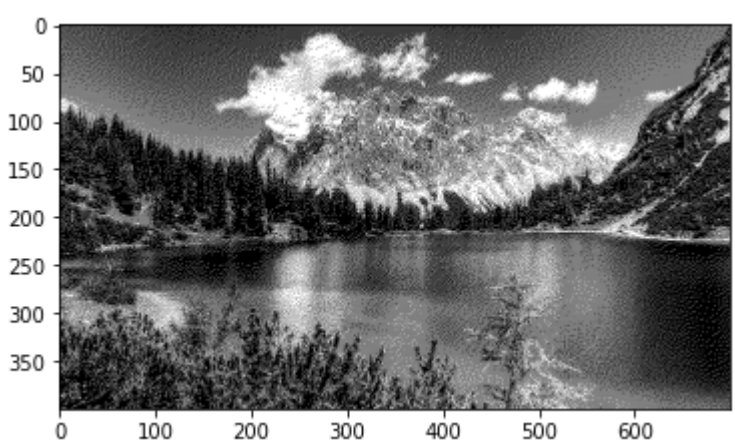
In [ ]:
```
H = block_filter(5)
print(H)
```

```
[[0.04 0.04 0.04 0.04 0.04]
 [0.04 0.04 0.04 0.04 0.04]
 [0.04 0.04 0.04 0.04 0.04]
 [0.04 0.04 0.04 0.04 0.04]
 [0.04 0.04 0.04 0.04 0.04]]
```

Next, we define the actual convolution operation. To prevent invalid indices at the border of the image, we introduce the padding **p**.

In [ ]:
```
def apply_filter(I, H):
    h, w = I.shape                          # image dimensions (height, width)
    n = H.shape[0]                          # filter size
    p = n // 2                              # padding size
    J = np.zeros_like(I)                    # output image, initialized with ze
ros

    for x in range(p, h-p):
        for y in range(p, w-p):
            J[x, y] = np.sum(I[x-p:x+n-p, y-p:y+n-p] * H)
    return J
```

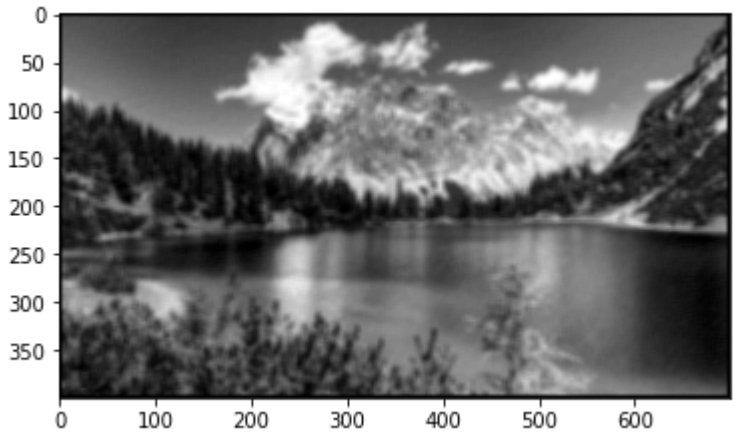
In [ ]:
```
image = Image.open('data/image.jpg')
image = image.convert('1') # convert image to black and white

image = np.array(image)

# image = np.zeros((200, 200), dtype=np.float)
# for x in range(200):
#     for y in range(200):
#         d = ((x-100)**2+(y-100)**2)**0.5
#         image[x, y] = d % 8 < 4

plt.imshow(image, cmap='gray',vmin=0.0, vmax=1.0)
plt.show()
```



In [ ]:
```
image = image.astype(float)
```

Next we test our implementation and apply a block filter with size 7

In [ ]:
```
n = 7
H = block_filter(n)
J = apply_filter(image, H)

plt.imshow(J, cmap='gray')
plt.show()
```



PyTorch Convolution

In [ ]:
```
from PIL import Image

img = Image.open('data/image.jpg')
img.thumbnail((256,256), Image.ANTIALIAS) # Resize to half to reduce the size
of this notebook.
```

In [ ]:
```
img
```

Out[ ]:

In [ ]:
```
import torch, torchvision
from torchvision import transforms
from torch import nn
```

In [ ]:
```
to_tensor = transforms.Compose([
   transforms.Grayscale(),  # Convert image to grayscale.
   transforms.ToTensor()    # Converts a PIL Image in the range [0, 255] to a
torch.FloatTensor in the range [0.0, 1.0].
])

to_pil = transforms.Compose([
    transforms.ToPILImage()
])
```

In [ ]:
```
input = to_tensor(img)
input.shape
```

Out[ ]:
```
torch.Size([1, 146, 256])
```

In [ ]:
```
to_pil(input)
```

Out[ ]:

2D convolution over an input image:

- `in_channels = 1` : an input is a grayscale image
- `out_channels = 1` : an output is a grayscale image
- `kernel_size = (3, 3)` : the kernel (filter) size is 3 x 3
- `stride = 1` : the stride for the cross-correlation is 1
- `padding = 1` : zero-paddings on both sides for 1 point for each dimension
- `bias = False` : no bias parameter (for simplicity)

In [ ]:
```
conv = nn.Conv2d(1, 1, (3, 3), stride=1, padding=1, bias=False)
```

In [ ]:
```
# The code below does not work because the convolution layer requires the dime
nsion for batch.
conv(input)
```

```
---------------------------------------------------------------------------
RuntimeError                              Traceback (most recent call last)
d:\Apps\machine-learning-projects\computer-vision\course\4_convolution.ipynb
Cell 25 in <module>
      <a href='vscode-notebook-cell:/d%3A/Apps/machine-learning-projects/comp
uter-vision/course/4_convolution.ipynb#X33sZmlsZQ%3D%3D?line=0'>1</a> # The c
ode below does not work because the convolution layer requires the dimension
for batch.
----> <a href='vscode-notebook-cell:/d%3A/Apps/machine-learning-projects/comp
uter-vision/course/4_convolution.ipynb#X33sZmlsZQ%3D%3D?line=1'>2</a> conv(in
put)

File c:\ProgramData\Anaconda3\envs\PhD\lib\site-packages\torch\nn\modules\mod
ule.py:1102, in Module._call_impl(self, *input, **kwargs)
   1098 # If we don't have any hooks, we want to skip the rest of the logic i
n
   1099 # this function, and just call forward.
   1100 if not (self._backward_hooks or self._forward_hooks or self._forward_
pre_hooks or _global_backward_hooks
   1101         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1102     return forward_call(*input, **kwargs)
   1103 # Do not call functions when jit is used
   1104 full_backward_hooks, non_full_backward_hooks = [], []

File c:\ProgramData\Anaconda3\envs\PhD\lib\site-packages\torch\nn\modules\con
v.py:446, in Conv2d.forward(self, input)
    445 def forward(self, input: Tensor) -> Tensor:
--> 446     return self._conv_forward(input, self.weight, self.bias)

File c:\ProgramData\Anaconda3\envs\PhD\lib\site-packages\torch\nn\modules\con
v.py:442, in Conv2d._conv_forward(self, input, weight, bias)
    438 if self.padding_mode != 'zeros':
    439     return F.conv2d(F.pad(input, self._reversed_padding_repeated_twic
e, mode=self.padding_mode),
    440                     weight, bias, self.stride,
    441                     _pair(0), self.dilation, self.groups)
--> 442 return F.conv2d(input, weight, bias, self.stride,
    443                 self.padding, self.dilation, self.groups)

RuntimeError: Expected 4-dimensional input for 4-dimensional weight [1, 1, 3,
3], but got 3-dimensional input of size [1, 146, 256] instead
```

We need to insert a dimension for a batch at dim=0.

In [ ]:
```
input = input.unsqueeze(0)
input.shape
```

Out[ ]:
```
torch.Size([1, 1, 146, 256])
```

In [ ]:
```
output = conv(input)
output.shape
```

Out[ ]:
```
torch.Size([1, 1, 146, 256])
```

Setting `padding=1` in the convolution layer, we obtain an image of the same size.

In [ ]:
```
output.shape
```

Out[ ]:
```
torch.Size([1, 1, 146, 256])
```

We need to remove the first dimension before converting to a PIL object.

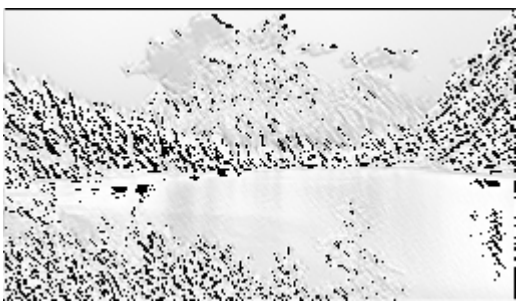
In [ ]:
```
output.data.squeeze(dim=0).shape
```

Out[ ]:
```
torch.Size([1, 146, 256])
```

Display the output from the convolution layer by converting `output` to a PIL object.

In [ ]:
```
to_pil(output.data.squeeze(dim=0))
```

Out[ ]:



Clip every value in the output tensor within the range of [0, 1].

In [ ]:
```
to_pil(torch.clamp(output, 0, 1).data.squeeze(dim=0))
```

Out[ ]:

In [ ]:
```
def display(img1, img2):
    im1 = to_pil(torch.clamp(img1, 0, 1).data.squeeze(dim=0))
    im2 = to_pil(torch.clamp(img2, 0, 1).data.squeeze(dim=0))
    dst = Image.new('RGB', (im1.width + im2.width, im1.height))
    dst.paste(im1, (0, 0))
    dst.paste(im2, (im1.width, 0))
    return dst
```

In [ ]:
```
display(input, output)
```

Out[ ]:

Identity

In [ ]:
```
conv.weight.data = torch.tensor([[[
    [0., 0., 0.],
    [0., 1, 0.],
    [0., 0., 0.],
]]])

output = conv(input)
display(input, output)
```

Out[ ]:

Brighten

In [ ]:
```
conv.weight.data = torch.tensor([[[
    [0., 0., 0.],
    [0., 1.5, 0.],
    [0., 0., 0.],
]]])
print(conv.weight.data)
output = conv(input)
display(input, output)
```

```
tensor([[[[0.0000, 0.0000, 0.0000],
          [0.0000, 1.5000, 0.0000],
          [0.0000, 0.0000, 0.0000]]]])
```

Out[ ]:

Darken

In [ ]:
```
conv.weight.data = torch.tensor([[[
    [0., 0., 0.],
    [0., 0.5, 0.],
    [0., 0., 0.],
]]])
print(conv.weight.data)
output = conv(input)
display(input, output)
```

```
tensor([[[[0.0000, 0.0000, 0.0000],
          [0.0000, 0.5000, 0.0000],
          [0.0000, 0.0000, 0.0000]]]])
```

Out[ ]:

Box blur

In [ ]:
```
conv.weight.data = torch.ones((1, 1, 3,3), dtype=torch.float) / 9.
print(conv.weight.data)
output = conv(input)
display(input, output)
```

```
tensor([[[[0.1111, 0.1111, 0.1111],
          [0.1111, 0.1111, 0.1111],
          [0.1111, 0.1111, 0.1111]]]])
```

Out[ ]:

Gaussian blur

In [ ]:
```
conv.weight.data = torch.tensor([[[
    [1., 2., 1.],
    [2., 4., 2.],
    [1., 2., 1.],
]]])/16.
print(conv.weight.data)
output = conv(input)
display(input, output)
```

```
tensor([[[[0.0625, 0.1250, 0.0625],
          [0.1250, 0.2500, 0.1250],
          [0.0625, 0.1250, 0.0625]]]])
```

Out[ ]:

Sharpen

In [ ]:
```
conv.weight.data = torch.tensor([[[
    [0., -1., 0.],
    [-1., 5., -1.],
    [0., -1., 0.],
]]])
print(conv.weight.data)
output = conv(input)
display(input, output)
```

```
tensor([[[[ 0., -1.,  0.],
          [-1.,  5., -1.],
          [ 0., -1.,  0.]]]])
```

Out[ ]:

In [ ]:
```
conv.weight.data = torch.tensor([[[
    [0., -2., 0.],
    [-2., 10., -2.],
    [0., -2., 0.],
]]])
print(conv.weight.data)
output = conv(input)
display(input, output)
```

```
tensor([[[[ 0., -2.,  0.],
          [-2., 10., -2.],
          [ 0., -2.,  0.]]]])
```

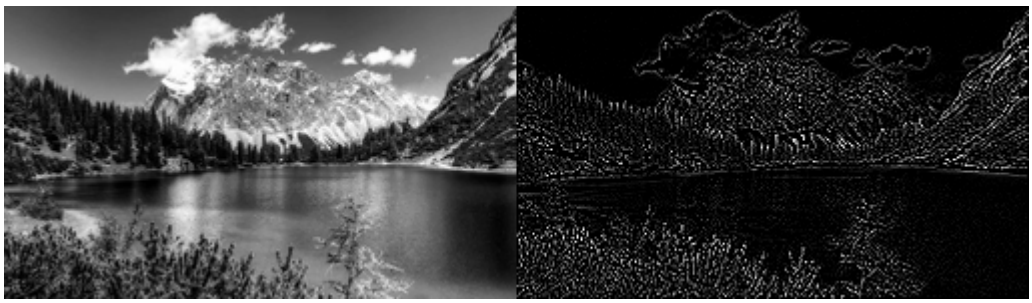
Out[ ]:

Edge detection

In [ ]:
```
conv.weight.data = torch.tensor([[[
    [0., 1., 0.],
    [1., -4., 1.],
    [0., 1., 0.],
]]])
print(conv.weight.data)
output = conv(input)
display(input, output)
```

```
tensor([[[[ 0.,  1.,  0.],
          [ 1., -4.,  1.],
          [ 0.,  1.,  0.]]]])
```

Out[ ]:



In [ ]:
```
conv.weight.data = torch.tensor([[[
    [-1., -1., -1.],
    [-1., 8., -1.],
    [-1., -1., -1.],
]]])
output = conv(input)
display(input, output)
```

Out[ ]:

In [ ]:
```
# TODO: Challenge, hard image with hard transformations
```

Maestria: Inteligencia artificial aplicada (MNA)

Materia:Visión computacional para imágenes y video

Profesor Titular: PhD. Gilberto Ochoa Ruiz.

Tarea: 3.2 Google Colab - Algoritmos de mejoramiento de imágenes en el dominio espacial.

Equipo: 5

Intengrantes y matricula:

- Denisse María Ramírez Colmenero -> A01561497
- Freddy Armendariz Herrera ->A01793672
- Samuel Elias Flores Gonzalez->A01793668
- Ernesto Nicanor Santillán Guerrero -> A01793675
- Juan Carlos López Cruz -> A01793052

In [ ]:
```
#Librerias a usar
from math import degrees, pi
import numpy as np
import argparse
import cv2
import os
import matplotlib.pyplot as plt
```

In [ ]:
```
# Acceso drive the google para leer  las imagenes
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call
drive.mount("/content/drive", force_remount=True).
```

Actividad 1:

Implementa los detectores de línea siguientes usando código en Python (es decir, sin usar librerías de OpenCV): Prewitt, Sobel y Laplaciano. Investiga la complejidad algoritmica de estos, ¿cual es más eficiente?

Visualizacion de imagenes

In [ ]:
```
def imagenes_vis(img,img_y,img_x,k):
  fig = plt.figure(figsize=(32, 10))
  # Se muestra imagen original.
  a=fig.add_subplot(1,4,1)
  plt.imshow(img,'gray')
  a.set_title('Original')
  # X
  a=fig.add_subplot(1,4,2)
  img_x_b= cv2.threshold(img_x, 70 ,255, cv2.THRESH_BINARY)
  plt.imshow(img_x_b[1],'gray')
  a.set_title(k+'(x)')
  # y
  a=fig.add_subplot(1,4,3)
  img_y_b= cv2.threshold(img_y, 70 ,255, cv2.THRESH_BINARY)
  plt.imshow(img_y_b[1], cmap='gray')
  a.set_title(k+'(y)')
  # Suma
  a=fig.add_subplot(1,4,4)
  img_s=img_y+img_x
  img_s_b= cv2.threshold(img_s, 70 ,255, cv2.THRESH_BINARY)
  plt.imshow(img_s_b[1], cmap='gray')
  a.set_title(k+'(suma x+y)')
```

Función de convolución.

In [ ]:
```
import cv2

import numpy as np

def convo(img,Kv,Kh, K_size=3):

    H, W = img.shape

    # Zero padding

    pad = K_size // 2

    out = np.zeros((H + pad * 2, W + pad * 2), dtype=np.float64)

    out[pad: pad + H, pad: pad + W] = img.copy().astype(np.float64)

    tmp = out.copy()

    out_v = out.copy()

    out_h = out.copy()

    # filtering

    for y in range(H):

        for x in range(W):

            out_v[pad + y, pad + x] = np.sum(Kv * (tmp[y: y + K_size, x: x + K
_size]))

            out_h[pad + y, pad + x] = np.sum(Kh * (tmp[y: y + K_size, x: x + K
_size]))

    out_v = np.clip(out_v, 0, 255)

    out_h = np.clip(out_h, 0, 255)

    out_v = out_v[pad: pad + H, pad: pad + W].astype(np.uint8)

    out_h = out_h[pad: pad + H, pad: pad + W].astype(np.uint8)

    return out_v, out_h
```

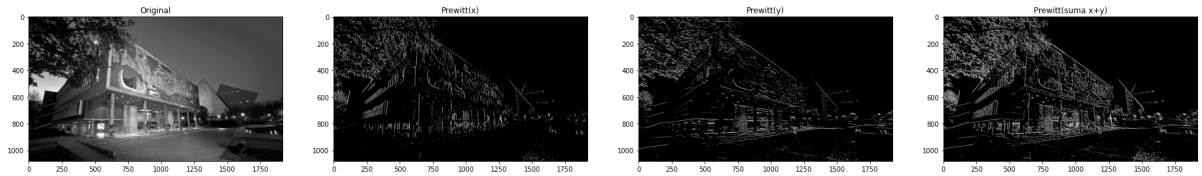
Prewitt

Kernel horizontal y kernel vertical para Prewitt

In [ ]:
```
## prewitt vertical kernel

    PKy = [[-1., -1., -1.],[0., 0., 0.], [1., 1., 1.]]

    ## prewitt horizontal kernel

    PKx =np.array ([[1, 0, -1],[1, 0, -1],[1, 0, -1]])
```

Visualización de imágenes

In [ ]:
```
img = cv2.imread("/content/drive/MyDrive/Img/Semana_3/tec.jpg",0)
out_py, out_px = convo(img,PKy,PKx,K_size=3) # Lectura de imagenes (Imagen ori
ginal, Kernel y, Kernel X, Tamaño de kernel=3)
imagenes_vis(img,out_py, out_px,'Prewitt')
```



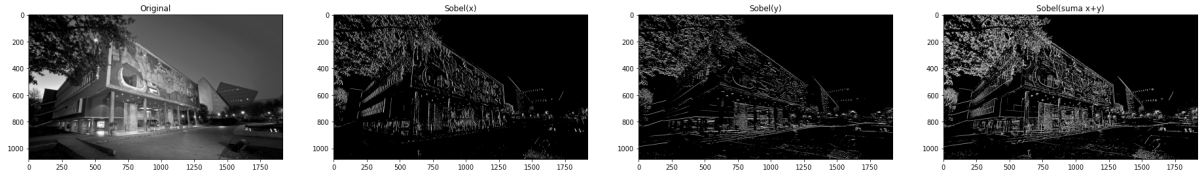
Sobel

Kernel horizontal y kernel vertical para Sobel

In [ ]:
```
#Sobel para el barrido horizontal
Gx = np.array([[-1, 0.0, 1.0], [-2.0, 0.0, 2.0], [-1.0, 0.0, 1.0]])
#Sobel para el barrido vertical
Gy = np.array([[-1, -2, -1.0], [0.0, 0.0, 0.0], [1.0, 2.0, 1.0]])
```

Visualización de imágenes

In [ ]:
```
img = cv2.imread("/content/drive/MyDrive/Img/Semana_3/tec.jpg",0)
out_sy, out_sx = convo(img,Gy,Gx,K_size=3) # Lectura de imagenes (Imagen origi
nal, Kernel y, Kernel X, Tamaño de kernel=3)
imagenes_vis(img,out_sy, out_sx,'Sobel')
```



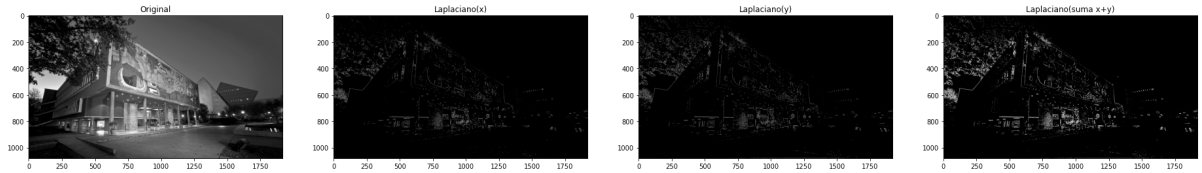
Laplaciano

Kernel horizontal y kernel vertical para Laplaciano

In [ ]:
```
#laplaciano positivo
Lpx = np.array([[0, 1, 0], [1, -4, 1], [0, 1, 0]])
#laplaciano negativo
Lpy = np.array([[0, -1, 0], [-1, 4, -1], [0, -1, 0]])
```

Visualización de imágenes

In [ ]:
```
img = cv2.imread("/content/drive/MyDrive/Img/Semana_3/tec.jpg",0)
out_ly, out_lx = convo(img,Lpy,Lpx,K_size=3) # Lectura de imagenes (Imagen ori
ginal, Kernel y, Kernel X, Tamaño de kernel=3)
imagenes_vis(img,out_ly, out_lx,'Laplaciano')
```



El operador de Prewitt es un detector de bordes que utilizando dos máscaras derivadas detecta tanto bordes verticales como horizontales. Funciona como una derivada de primer orden y calcula la diferencia de la intensidad entre los pixeles en la región del borde. Los valores originales de los pixeles en la imágen son cambiados en la columna/fila central de su máscara por ceros, lo que hace que la intensidad en el borde aumente y por lo tanto, que se vean más claros los limites.

El operador Sobel es muy similar al operador Prewitt. Es también de primer orden y utiliza las dos máscaras para los bordes verticales y horizontales. La única diferencia es que los coeficientes en la máscara no son fijos y pueden utilizarse diferentes valores según convenga. De esta manera, al utilizar más peso en la máscara, la intensidad de los bordes se incrementa.

Por último, el operador Laplaciano es una máscara derivada de segundo orden y detecta bordes exteriores (operador positivo) y bordes hacia adentro o internos (operador negativo). Estos operadores generan bordes grisáceos y discontinuidades sobre un fondo oscuro, produciendo así bordes hacia adentro y hacia afuera en una imagen.

El operador Laplaciano es de mayor grado que Prewitt y Sobel por lo que tiene una complicación algoritmica más elevada pero para implementación se puede decir que los tres son sencillos. Para esta imágen en específico el operador que mejor resultado arrojó fue el Sobel, aunque el operador Prewitt también tuvo buen desempeño. El operador Laplaciano si genero bordes pero no tan notorios.

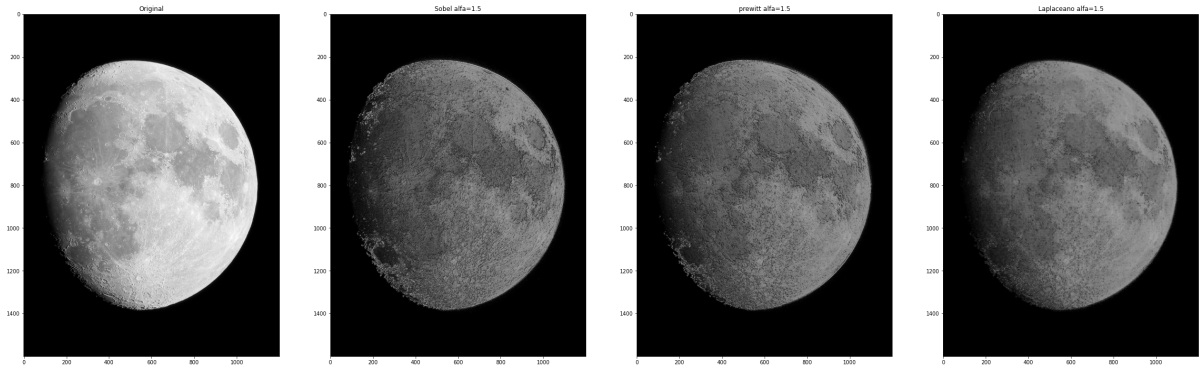
Actividad 2:

Implementa un algoritmos de realce o mejoramiento de imágenes mediante un algoritmo de en el cual se extraen las líneas de la imagen y después se aplica la diferencia con la imagen original, multiplicando los píxeles de la imagen “máscara” (las líneas encontradas) por un factor alfa mayor a 1.

In [ ]:
```
def imagenes_vis_realce(img,img_s,img_p,img_l,alfa):
  fig = plt.figure(figsize=(40, 40))
  # Se muestra imagen original.
  a=fig.add_subplot(1,4,1)
  plt.imshow(img,'gray')
  a.set_title('Original')
  # Sobel
  a=fig.add_subplot(1,4,2)
  plt.imshow(img_s,'gray')
  a.set_title("Sobel alfa="+str(alfa))
  # Prewitt
  a=fig.add_subplot(1,4,3)
  plt.imshow(img_p, 'gray')
  a.set_title('prewitt alfa='+str(alfa))
  # Laplaceano
  a=fig.add_subplot(1,4,4)
  plt.imshow(img_l, 'gray')
  a.set_title('Laplaceano alfa='+str(alfa))
```

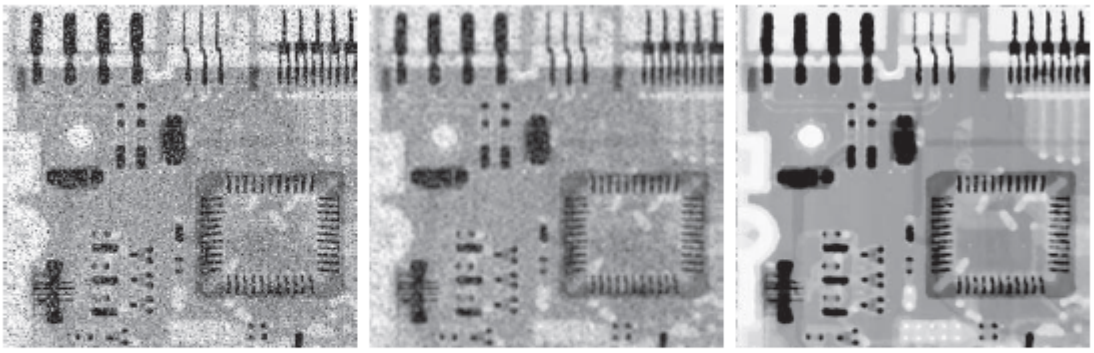
In [ ]:
```
img_2 = cv2.imread("/content/drive/MyDrive/Img/Semana_3/Luna_mineral.jpg",0)
lineas_sobel=sum(convo(img_2,Gy,Gx,K_size=3))
lineas_prewitt=sum(convo(img_2,PKy,PKx,K_size=3))
lineas_laplaceano=sum(convo(img_2,Lpy,Lpx,K_size=3))
```

In [ ]:
```
alpha=1.5
realce_sobel=abs(img_2-(lineas_sobel)*alpha)
realce_prewitt=abs(img_2-(lineas_prewitt)*alpha)
realce_laplaceano=abs(img_2-(lineas_laplaceano)*alpha)
imagenes_vis_realce(img_2,realce_sobel,realce_prewitt,realce_laplaceano,alpha)
```



Actividad 3:

Buscar una aplicación médica (en el libro de Gonzalez viene varios ejemplos de imágenes PET) y hacer mejoramiento usando la técnica de la Figura 3.43 combinando diferentes etapas de procesamiento de imagenes.



a b c

FIGURE 3.43 (a) X-ray image of a circuit board, corrupted by salt-and-pepper noise. (b) Noise reduction using a 19×19 Gaussian lowpass filter kernel with $\sigma = 3$. (c) Noise reduction using a 7×7 median filter. (Original image courtesy of Mr. Joseph E. Pascente, Lixi, Inc.)

Aplicación:

Eliminación de calaveras y fondos

- El cráneo y el tejido blando circundante pueden contribuir a ruido en el estudio, cuando el ROI se encuentra solo dentro del cráneo cavidad. En tales escenarios, se puede realizar la extracción del cráneo y la eliminación del fondo. La intensidad de HU de las estructuras óseas es significativamente mayor que la de otras estructuras y, por lo tanto, se puede suprimir fácilmente al establecer los valores de píxeles por encima de un umbral determinado en un valor mínimo (p. ej., 0)
- Después de quitar el cráneo y el fondo, podría haber ruido dentro de la región del cerebro.

Funcion para ver imagenes.

In [ ]:
```
def imagenes_vis_noise(img,img_g,img_m):
  fig = plt.figure(figsize=(32, 10))
  # Se muestra imagen original.
  a=fig.add_subplot(1,3,1)
  plt.imshow(img,'gray')
  a.set_title('Original')
  # Algoritmo
  a=fig.add_subplot(1,3,2)
  plt.imshow(img_g,'gray')
  a.set_title('Gaussian low pass filter')
  # Open
  a=fig.add_subplot(1,3,3)
  plt.imshow(img_m, cmap='gray')
  a.set_title('Median Filter')
```

Lectura de imagen.

In [ ]:
```
#img = cv2.imread("/content/drive/MyDrive/Img/Semana_3/Circuit Board.jpg",0)
img_3 = cv2.imread("/content/drive/MyDrive/Img/Semana_3/Tomografia.jpg",0)
```

Noise reduction using a 19 × 19 Gaussian lowpass filter kernel with s = 3.

Algoritmo.

In [ ]:
```
# Gauss filter
def gfunc(x,y,sigma):
    return (np.exp(-(x**2 + y**2)/(2*(sigma**2))))/(2*3.14*(sigma**2))

def gaussFilter(size, sigma):
    out = np.zeros(size)
    for i in range(size[0]):
        for j in range(size[1]):
            out[i,j] = gfunc(i-size[0]//2,j-size[1]//2, sigma )
    return out/np.sum(out)
```

In [ ]:
```
gaussianFilter = gaussFilter((19,19),3) # Kernel nxn, sigma =
Gauss_A, basura = convo(img_3,gaussianFilter,0,K_size=19) # Se utiliza el algo
ritmo de convolucion para usar la matriz que se genero con el filtro Gausiano.
```

Codigo en Open CV.

In [ ]:
```
Gauss_CV = cv2.GaussianBlur(img_3,(19,19),3)
```

Noise reduction using a 7 × 7 median filter.

Algoritmo.

In [ ]:
```
def median_filter(data, filter_size):
    temp = []
    indexer = filter_size // 2
    data_final = []
    data_final = np.zeros((len(data),len(data[0])))
    for i in range(len(data)):

        for j in range(len(data[0])):

            for z in range(filter_size):
                if i + z - indexer < 0 or i + z - indexer > len(data) - 1:
                    for c in range(filter_size):
                        temp.append(0)
                else:
                    if j + z - indexer < 0 or j + indexer > len(data[0]) - 1:
                        temp.append(0)
                    else:
                        for k in range(filter_size):
                            temp.append(data[i + z - indexer][j + k - indexe
r])

            temp.sort()
            data_final[i][j] = temp[len(temp) // 2]
            temp = []
    return data_final
```

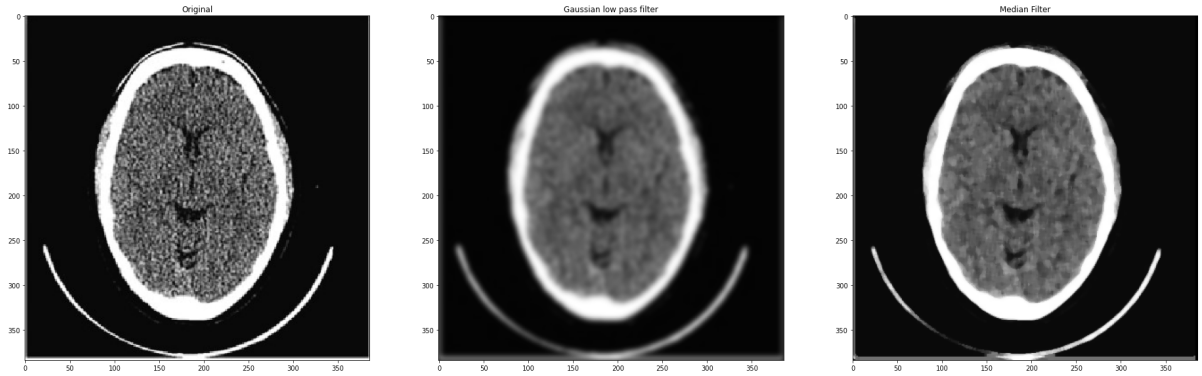
In [ ]:
```
median_A = median_filter(img_3, 7)
```

Codigo en Open CV.

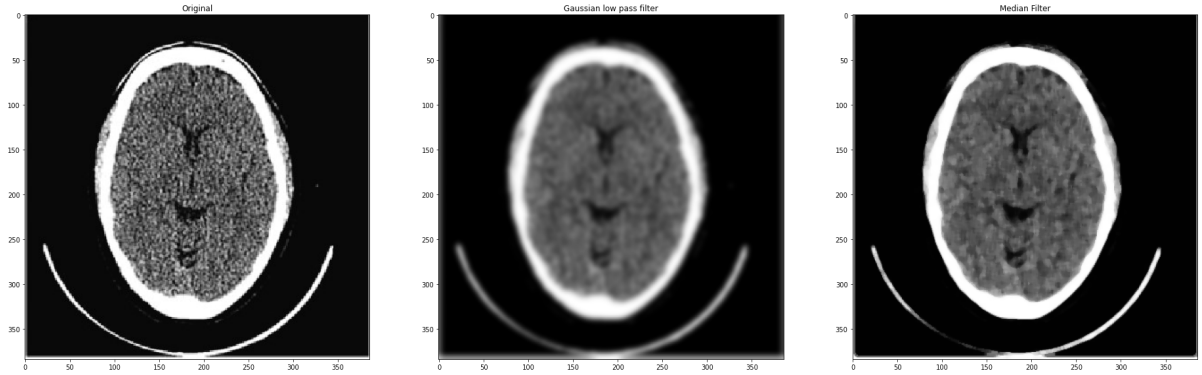
In [ ]:
```
median_CV= cv2.medianBlur(img_3,7)
```

Visualizacion de imagenes

In [ ]:
```
imagenes_vis_noise(img_3,Gauss_A,median_A)
```



In [ ]:
```
imagenes_vis_noise(img_3,Gauss_CV,median_CV)
```



Bibliografía:

- Niharika Krishnan, Canny Edge Detector (2021), GitHub repository, https://github.com/niharikakrishnan/Canny-Edge-Detector/blob/main/canny.py (https://github.com/niharikakrishnan/Canny-Edge-Detector/blob/main/canny.py)
- Feature Detectors - Sobel Edge Detector. (s. f.). https://homepages.inf.ed.ac.uk/rbf/HIPR2/sobel.htm (https://homepages.inf.ed.ac.uk/rbf/HIPR2/sobel.htm)
- Create Laplacian Kernel — v5.3.0. (s. f.). https://examples.itk.org/src/core/common/createlaplaciankernel/documentation (https://examples.itk.org/src/core/common/createlaplaciankernel/documentation)
- Vikas Gola, Image Filtering (2020), GitHub repository, https://github.com/vikasgola/image-filtering (https://github.com/vikasgola/image-filtering)
- Metehan Çetinkaya,Python Median Filter (2018), GitHub repository, https://github.com/MeteHanC/Python-Median-Filter/blob/master/MedianFilter.py (https://github.com/MeteHanC/Python-Median-Filter/blob/master/MedianFilter.py)
- Operador Laplaciano. (2020b, diciembre 10). Stack. https://isolution.pro/es/t/dip/laplacian-operator/operador-laplaciano (https://isolution.pro/es/t/dip/laplacian-operator/operador-laplaciano)
- Prabhu Kavin, Balasubramanian & Sannasi, Ganapathy & Suthanthiramani, Pradeep & Arputharaj, Kannan. (2020). A modified digital signature algorithm to improve the biomedical image integrity in cloud environment. 10.1016/B978-0-12-820024-7.00013-X.